CO3-ASSESSMENT TOOL –2

CSA0607-Design and Analysis of Algorithms

Course Outcome: CO3_AT2

Student Name: A.Sai Joshitha(192424220)

Q11. Case Study: Social Media Feed Sorting System (Merge Sort)

A social media platform must sort millions of posts based on timestamps and relevance to ensure timely content delivery. Analyze how Merge Sort can be applied to handle large-scale data efficiently while maintaining stability. Identify challenges such as real-time updates and scalability. Apply divide and conquer principles to explain the sorting mechanism. Analyze time complexity and design a suitable solution, justifying why Merge Sort is appropriate.how to do this execution

Problem Identification

Social media platforms such as Instagram, Facebook, and X (Twitter) receive millions of posts every minute. To display the latest and most relevant posts quickly, the platform needs an efficient sorting algorithm. This case study implements Merge Sort to organize posts based on their timestamps while preserving the order of posts with equal timestamps.

Objectives

- Develop a social media feed sorting application.
- Implement the Merge Sort algorithm.
- Sort posts based on timestamps.
- Demonstrate the Divide and Conquer technique.
- Analyze the algorithm's performance**.**

System Design

Input

Post ID

User Name

Timestamp

Relevance Score

↓

Merge Sort

- Divide the list
- Sort each half
- Merge into a sorted list

↓

Output

Sorted Social Media Feed

Algorithm

1. Start the program.
2. Create a list of social media posts.
3. Divide the list into two halves.
4. Continue dividing until each list contains one post.
5. Merge the lists by comparing timestamps.
6. Display the sorted feed.
7. Stop.

Code

```
class Post:
    def __init__(self, post_id, user, timestamp, relevance):
        self.post_id = post_id
        self.user = user
        self.timestamp = timestamp
        self.relevance = relevance
    def __str__(self):
        return f"{self.post_id} | {self.user} | {self.timestamp} | {self.relevance}"
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].timestamp <= right[j].timestamp:
```

```
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def merge_sort(posts):
    if len(posts) <= 1:
        return posts
    mid = len(posts) // 2
    left = merge_sort(posts[:mid])
    right = merge_sort(posts[mid:])
    return merge(left, right)
posts = [
    Post(101, "Alice", 1045, 80),
    Post(102, "Bob", 1010, 95),
    Post(103, "Charlie", 1100, 70),
    Post(104, "David", 950, 85),
    Post(105, "Emma", 1030, 90)
]
print("====== Social Media Feed ======")
print("\nPosts Before Sorting")
print("ID | User | Timestamp | Relevance")
for post in posts:
    print(post)
sorted_posts = merge_sort(posts)
print("\nPosts After Sorting")
```

print("ID | User | Timestamp | Relevance")

for post in sorted_posts:

    print(post)

main.py

Share Run

```
class Post:
    def __init__(self, post_id, user, timestamp, relevance):
        self.post_id = post_id
        self.user = user
        self.timestamp = timestamp
        self.relevance = relevance

    def __str__(self):
        return f"{self.post_id} | {self.user} | {self
            .timestamp} | {self.relevance}"


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].timestamp <= right[j].timestamp:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
```

OUTPUT:

Output

```
====== Social Media Feed ======

Posts Before Sorting
ID | User | Timestamp | Relevance
101 | Alice | 1045 | 80
102 | Bob | 1010 | 95
103 | Charlie | 1100 | 70
104 | David | 950 | 85
105 | Emma | 1030 | 90

Posts After Sorting
ID | User | Timestamp | Relevance
104 | David | 950 | 85
102 | Bob | 1010 | 95
105 | Emma | 1030 | 90
101 | Alice | 1045 | 80
103 | Charlie | 1100 | 70

=== Code Execution Successful ===
```

Time Complexity:

Case	Time Complexity	Explanation
Best Case	**O(n log n)**	The list is divided into halves and merged efficiently, regardless of the initial order.
Average Case	**O(n log n)**	Merge Sort always divides the list into equal halves and performs merging in linear time.
Worst Case	**O(n log n)**	Even if the data is in reverse order, Merge Sort maintains the same performance.

Space Complexity:

Space Used	Complexity	Explanation
Auxiliary Space	O(n)	An additional temporary array/list is required during the merge process.
Recursion Stack	O(log n)	Recursive function calls require stack space proportional to the height of the recursion tree.

Conclusion:

The Social Media Feed Sorting System successfully demonstrates the implementation of the Merge Sort algorithm for sorting posts based on timestamps. By using the Divide and Conquer approach, Merge Sort provides efficient and stable sorting with a time complexity of **O(n log n)**. This makes it well suited for large-scale social media platforms that require fast and reliable feed organization.